CS 409/6011 : Introduction to Cryptography Fall 2026

Lab 1: One-Time Pad and Variations, PRGs, Modes of Operation

Instructor: Sruthi Sekar *TAs:* Ragav, Sagnik, Samipa, Divya, Harsha

Challenge 1: Two-Time Pad

Author: Nilabha Saha | **Theme:** One-Time Pad

The one-time pad, as you have learnt, is perfectly secure. However, its perfect secrecy depends on the key being used exactly once. If the same key is reused, the resulting "two-time pad" is no longer secure as XORing the two ciphertexts gives you the XOR of the plaintexts, which is a lot of information about the two messages. In fact, if the messages are in natural language, it may even allow you to extract both the messages fully given their XOR. In this challenge, you'll do exactly that.

`ciphertext1.enc` and `ciphertext2.enc` contain two one-time pad encrypted ciphertexts encrypted with the same key. One of them is a "flag" used in a Capture The Flag competition (hint: The format of the flag is "*cs409*{...}" and the other one is a normal English sentence. Use the redundancy of the English language to figure out the entire flag. Feel free to look at `encrypt.py` to understand how to use the `strxor` function to XOR two byte objects in python.

Historical Note: *During WWII, the USSR made the mistake of using several one-time pad keys twice and the US intelligence agencies used this blunder to their advantage to decrypt the Soviet secret messages. Look up Venona Project if you're interested in the history!*

Challenge 2: One-Time-And

Author: Harshit Somani | **Theme:** One-Time Pad

Alice, a cryptography expert at a massive company decides to encrypt confidential data using one time pad to distribute to the employees.

She uses several secret keys to encrypt her message, giving her several ciphers that she can distribute. These ciphers are stored in `ciphers.dat` in **hexadecimal** format.

However, she was working while eating popcorn and her buttery fingers slipped, leading to a typo: instead of the bitwise XOR (`^`) operator, she accidentally used the bitwise AND (`&`) operator.

You get your hands on the `ciphers.dat` file and the encryption algorithm `encrypt.py`. Use these files to exploit the vulnerability and decrypt Alice's confidential data!

Challenge 3: PRGone Wrong

Author: Ravi Prakash | **Theme:** PRG

A pseudorandom generator (PRG) should produce output that appears random and, more importantly, should not allow an attacker to recover its seed from the output. You are given:

- prg.py - the implementation of the PRG
- output.txt - output produced by the PRG
- The flag format: cs409{...}

The PRG provided in this challenge is faulty. Your task is to recover the original seed and obtain the flag.

Note: In the standard formulation of a PRG, the seed is usually modeled as a uniformly random string. As you might notice, the seed used in this challenge is a structured value (the flag). However, a structured seed does not automatically break a PRG's output pseudorandomness if the construction is specifically designed to tolerate that structure. See, for example, this paper: Structured-Seed Local Pseudorandom Generators Therefore, do not assume that the structure of the seed is the vulnerability in this challenge. Your task is to identify the actual weakness in the PRG construction.

Challenge 4: The Catastrophic Equality

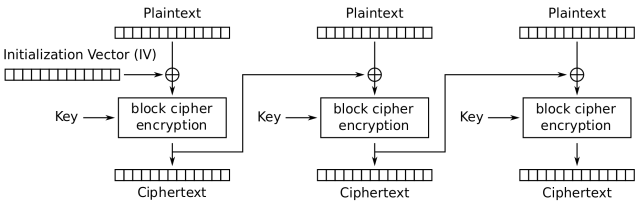
Author: Nilabha Saha | **Theme: Block Cipher (CBC)**

Some people believe that since the key is already being randomly generated, the key can also be repurposed as the IV as long as the IV is not being revealed. This is a rookie mistake and in general, a very bad idea.

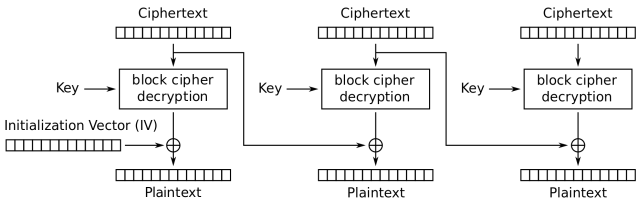
In this challenge, you are going to submit "parameters" (each message is a string of the form *key1*=*value1*`&`*key2*=*value2*`&`$\cdots$`&`*key-n*=*value-n*) to a server. These messages are supposed to comprise of normal low-ASCII characters and should not contain the string `admin=true` as a substring. You will receive the encryption of your messages. The server also lets you send encrypted parameters to it, which it will decrypt internally; while the server does not send you the decryption, it will throw an error if it finds offending invalid characters in the decrypted text. If you can somehow send the server an encrypted parameter string which decrypts to a normal string which has `admin=true` as a substring, you will receive the encryption of the flag in return.

Abuse the fact that the server repurposes the key as the IV to craft an intelligent payload which allows you to recover the flag. Good luck on your mission!

The following diagrams could be of help to you (here "encryption" and "decryption" refer to the forward and reverse applications of the block cipher):



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

Challenge 5: Never Painted by the Numbers

Author: Nilabha Saha | **Theme: CTR Mode**

The CTR (CounTeR) mode of encryption is commonly used to convert a block cipher into an equivalent stream cipher. Do take a look at `https://pycryptodome.readthedocs.io/en/latest/src/cipher/classic.html#ctr-mode` to understand how to use the PyCryptodome library for CTR encryption mode (along with an explanation of how CTR mode works if you need a refresher).

In this challenge, you will interact with a server which essentially acts as an echo server - it outputs back to you exactly what you input to it – except that the response is encrypted. However, if you input `!flag` to it, the server instead outputs the flag to you – again, encrypted.

The encryption is done using CTR mode, using a key you don't know. But you suspect that the implementation of the CTR mode encryption in the server has some vulnerability. Refer to the `server.py` template file for details on the workings of the server, identify the vulnerability and exploit it to retrieve the flag.

Bonus Challenge: Canis Lupus Familiaris

Author: Nilabha Saha | **Theme: Padding Oracle (CBC)**

A padding oracle attack is a cryptographic attack that targets the padding scheme used in block cipher modes, particularly in Cipher Block Chaining (CBC) mode.

In practice, padding is often added to plaintext messages to ensure that the length of the message is a multiple of the block size of the cipher being used. A padding oracle is a term used to describe a vulnerability that arises when an attacker can determine whether a given ciphertext has a valid padding or not. The padding is typically added to ensure that the plaintext can be properly aligned into blocks before encryption. The padding scheme we often use is PKCS#7.

The padding oracle attack can be extremely dangerous in the sense that one could completely recover the plaintext given access to a padding oracle. In fact, that is what you will do in this challenge. You can refer to this webpage to learn about the technical workings of the padding oracle attack. You would be familiar with a lot of the material in the initial part of the webpage, so you can directly start reading from the section titled "Cipher-block chaining (CBC)".

In this challenge, you will interact with a server which will act as a padding oracle. In `solution_template.py`, you can make use of the function `validate_padding` as a blackbox for server communication (refer to the script comments for further details).

Historical Note: *The padding oracle attack forms the basis for a nasty attack, famously referred to in the field of network security as the POODLE attack. POODLE, which stands for "Padding Oracle On Downgraded Legacy Encryption", is a security vulnerability that affects the widely used SSL (Secure Sockets Layer) and early TLS (Transport Layer Security) protocols. This attack was discovered in 2014 and is a serious threat to the security of encrypted communications. The primary consequence of POODLE is the potential disclosure of sensitive information. By successfully executing the attack, an attacker can decrypt and access the contents of encrypted communication, including login credentials, personal information, and other sensitive data.*
Web browsers often support multiple SSL/TLS versions to maintain compatibility with various websites. POODLE can be particularly problematic for browsers, as attackers can exploit the vulnerability to compromise the security of connections made by the browser.
The discovery of POODLE accelerated the process of deprecating and phasing out support for SSL 3.0 across the industry. Subsequent versions of SSL and TLS, such as TLS 1.0, TLS 1.1 and TLS 1.2, have also been deprecated due to security vulnerabilities.
